

Introduzione al Python Notebook

Link:

https://kiwi.app/notes/note_01m3ms5v66eh7b9katy97dx52p

Strumenti e ambiente di lavoro

- Python Notebook: un formato che mescola codice e testo, utile per eseguire esempi e inserire annotazioni grafiche; non è un "notebook" hardware ma un ambiente software interattivo.
- Visual Studio Code (VS Code): editor consigliato per aprire e lavorare con i Python notebook (molto usato, acquistato/sviluppato da Microsoft); migliora la visualizzazione e l'uso dei notebook rispetto ad altri editor.
- Anaconda: distribuzione consigliata per installare Python su computer personali; è utile installarla in anticipo anche se non si conosce ancora tutto di Python.

Organizzazione del corso e valutazione

- Iscrizione a Classroom per materiali e comunicazioni.
- Laboratorio e simulazioni offrono bonus di punteggio per l'esame finale.
- Ci sono più docenti coinvolti: saranno presentati in aula (es. Daniele Alteni).

Concetti base: cos'è un programma e la CPU

- Programma: una sequenza ordinata di istruzioni eseguite dalla CPU (il "cervello" del computer) per ottenere un risultato; analogo a istruzioni di montaggio (es. manuale Ikea): ordine e passaggi contano.
- Scrivendo programmi in Python si dà istruzioni alla CPU; con il tempo si possono costruire programmi molto semplici fino a programmi grandi e complessi.

Tipi numerici e rappresentazione dei numeri

Interi (int)

- Si scrivono come una sequenza di cifre (es. 17).

- È possibile usare l'underscore _ per migliorare la leggibilità di numeri grandi (es. 1_000_000).
- Internamente memorizzati in binario; Python supporta interi di lunghezza praticamente illimitata.

Numeri in virgola mobile (float) 🟢

- Rappresentano i numeri con la virgola (decimali); hanno limiti sia nella precisione che nell'intervallo.
- Rappresentazione in floating point: combinazione di segno, esponente e mantissa. La formula generale è: $[(-1)^{\text{sign}} \times 2^{\text{exponent} - \text{offset}} \times (1.\text{mantissa})]$
- Esempio: una configurazione di bit può rappresentare il numero -6.75 usando la conversione segno/esponente/mantissa (procedura di calcolo spiegata con i bit).
- Standard comune usa una mantissa di 52 bit (double precision), che corrisponde a circa 15–16 cifre decimali: il controllo tipico è $[10^{15} < 2^{52} < 10^{16}]$
- Limiti: non tutti i numeri decimali possono essere rappresentati esattamente; attenzione a errori di arrotondamento nelle operazioni numeriche.

Conversione interno/esterno

- Gli umani leggono numeri in base 10; il computer li memorizza in binario. Python gestisce automaticamente le conversioni tra rappresentazioni visuali e internamente in bit.

Operatori aritmetici e priorità degli operatori ➗ ✖

- Moltiplicazione: *
- Divisione: / (restituisce float)
- Divisione intera: // (restituisce la parte intera)
- Modulo (resto): %
- Potenza: ** (es. $10^{**}2$ è 100)

Esempi e priorità: Python applica la precedenza: **esponenziazione prima, poi moltiplicazione/divisione/modulo nell'ordine in cui sono incontrati, poi addizione/sottrazione**. Esempio: $3 + 14 * 5 / 4$ viene valutato come $3 + ((14 * 5) / 4)$ e dà 20.5. * Per forzare un ordine diverso usare le **parentesi tonde**: solo le parentesi tonde cambiano l'ordine di esecuzione; le graffe e le quadre hanno altri significati in Python.

Tipi booleani (bool)

- Due soli valori: True e False (con la T e la F maiuscole).
- Usati nelle condizioni per decidere flusso e controllo del programma.
- Conversione con bool(x): bool(0) e bool(0.0) → False
- bool(1), bool(2), bool(-42), bool(0.1) → True (qualsiasi numero diverso da zero è True)

Tipo: **bool** (diverso da `int`, `float`, `str`).

Mantiene solo due stati: è un insieme finito di cardinalità 2.

Stringhe (str)

- Stringa (str): sequenza di caratteri (lettere, cifre, spazi, simboli) racchiusa tra virgolette ('...' o "...").
- Possono contenere testi di qualunque lunghezza (una frase, una pagina, un libro).
- Internamente ogni carattere ha un indice: l'indicizzazione parte da 0 (il primo carattere è posizione 0, il secondo 1, ecc.).
- Esempio: "Paperino andò al mare" è una stringa; Python la riconosce come testo e la stampa così com'è.

Funzione type e identificazione dei tipi

- type(x) restituisce il tipo dell'oggetto x. type(17) → int
- type(17.5) → float
- type(True) → bool
- type("ciao") → str

Utile per capire che cosa Python ha interpretato nella parentesi (intero, float, stringa, booleano).

Tabella dei tipi principali

Tipo	Descrizione
int	Numeri interi, precisione arbitraria (es. 17, -42)
float	Numeri in virgola mobile, rappresentazione in floating point (es. 17.5)
bool	Valori booleani: True, False (usati per condizioni)
str	Stringhe: sequenze di caratteri racchiuse tra virgolette (es. "ciao")

Note pratiche e suggerimenti

- Installare Anaconda e VS Code sul proprio computer per sperimentare i notebook.
- Usare underscore per rendere leggibili numeri grandi.
- Ricordare sempre la distinzione tra int (illimitato) e float (limitato e soggetto a errori di rappresentazione).
- Prestare attenzione a True/False (case-sensitive) quando si scrivono condizioni.

Stringhe e caratteri

- Stringa: una sequenza di caratteri memorizzata come testo tra virgolette, ad esempio "paperino andò al mare". Le stringhe servono a memorizzare testi di qualunque lunghezza.
- Carattere: qualsiasi simbolo della stringa (lettera, cifra, spazio, punteggiatura, accento, simbolo). Nei linguaggi di programmazione si usa il termine generico carattere per includere tutto questo.

Termine	Descrizione
stringa	Sequenza di caratteri tra virgolette.
carattere	Singolo simbolo all'interno della stringa (incluso lo spazio).

Python memorizza internamente ogni carattere come un numero (codifica) e quindi la stringa è trattata come una sequenza di elementi indirizzabili.

Indici e numerazione

- Gli elementi di una stringa sono numerati con indici che partono da 0 (il primo carattere ha indice 0). Questo vale in quasi tutti i contesti di programmazione.

- Es.: per la stringa "paperino" gli indici sarebbero:

Indice	Carattere
0	p
1	a
2	p
3	e
4	r
5	i
6	n
7	o

- Esiste anche una numerazione negativa: l'ultimo carattere ha indice -1, il penultimo -2, ecc. Le due numerazioni (positiva da sinistra e negativa da destra) convivono e si possono usare entrambe in Python. Lo zero non è parte della scala negativa perché sarebbe ambigua.

Variabili e assegnazione



- Assegnazione: l'operazione che mette un valore (numero, stringa, booleano, ecc.) in una variabile. Sintassi tipica: una sequenza di caratteri (nome della variabile, senza virgolette) a sinistra dell'uguale, seguito dal valore a destra. Esempio: A = "paperino"

Sintassi vs Semantica:

- Sintassi: come si scrive l'istruzione (es. nome = valore).
- Semantica: cosa significa/che effetto ha l'istruzione (es. memorizzare il valore nella cella etichettata dal nome).

Visualizzazione concettuale: una variabile è come una scatola etichettata (es. A) che contiene il valore (es. la stringa "paperino").

Nomi di variabile e parole riservate ⚠

- Il nome della variabile deve rispettare regole (sequenza di caratteri legittima) e non può coincidere con parole riservate o nomi di funzioni built-in (es. non usare ord o type come nome di variabile), altrimenti si crea conflitto con la sintassi/semantica del linguaggio.

Indicizzazione e slicing (A[5]) ✂

- Sintassi base: `nome_variabile[indice]` — le parentesi quadre con un intero indicano l'estrazione di un singolo carattere.
- Semantica: restituisce il carattere presente nella stringa memorizzata nella variabile, alla posizione indicata dall'indice. Esempio: se `A` contiene "paperino", allora `A[5]` restituisce 'i'.
- L'indice può essere positivo (conta da sinistra, a partire da 0) o negativo (conta da destra, con -1 = ultimo carattere).

ord() e chr() — conversione tra carattere e codice

Funzione	Sintassi	Descrizione	Esempio
<code>ord()</code>	<code>ord('c')</code>	Riceve un singolo carattere e restituisce il suo codice numerico (codifica Unicode/ASCII).	<code>ord('P') → 80</code>
<code>chr()</code>	<code>chr(105)</code>	Riceve un intero e restituisce il corrispondente carattere secondo la tabella di codifica.	<code>chr(105) → 'i'</code>

- Vincoli: `ord` richiede un singolo carattere (non una stringa pluricarattere). `chr` richiede un intero valido nel range della tabella.
- Internamente i caratteri sono memorizzati come sequenze di bit; le tabelle standard (es. ASCII, Unicode) associano a ogni carattere un numero che poi viene rappresentato in binario dal computer.

Perché convertire caratteri in numeri?

- Trattare i caratteri come numeri permette confronti e ordinamenti coerenti (es. ordinare parole secondo i codici dei caratteri);
- applicare algoritmi numerici ai testi (utile in sorting, ricerca, e data science);
- manipolazioni bit-level e conversioni di codifica.

Note pratiche e dettagli utili

- Lo slicing più avanzato (intervalli, step, ecc.) ha una sintassi estesa (es. `a[start:stop:step]`) che approfondisce l'estrazione di sottostringhe; qui è stato introdotto il caso elementare `nome[indice]`.
- Evitare di ridefinire nomi di funzioni built-in (come `ord`, `chr`, `type`) per non rompere funzionalità.

- Usare indici negativi è comodo quando si vuole lavorare dal fondo della stringa (feature particolarmente comoda in Python).

Operazioni base sulle stringhe

- Concatenazione (addizione): l'operatore + tra due stringhe le unisce in ordine. Ad esempio `s1 + s2` con `s1 = "Paperino"` e `s2 = " va al mare"` produce `"Paperino va al mare"`. La concatenazione si estende a una somma di n stringhe per ottenere una singola stringa risultante con i caratteri nella stessa sequenza.
- Moltiplicazione: una stringa moltiplicata per un intero positivo k ripete la stringa k volte (es. `"ha" * 3` → `"hahaha"`). Queste operazioni permettono di trattare il testo come sequenze di caratteri manipolabili numericamente.

Delimitatori: apici singoli e doppi

- Entrambi i tipi sono validi: una stringa può essere delimitata con apice singolo ('...') o virgolette doppie ("..."). Internamente l'interprete può rappresentare la stringa con uno dei due tipi; ciò non influisce sul contenuto.
- Quando scegliere: se la stringa contiene un apostrofo (es. `Paperina è...`) conviene delimitare la stringa con virgolette doppie per evitare ambiguità; viceversa, se contiene virgolette doppie, usare apici singoli.
- Caso più complesso — presenza di entrambi: se la stringa include sia apici singoli che doppi all'interno, si risolve usando l'escape con il backslash \ prima del carattere ambiguo (es. `"L'ombrello di \"Paverino\" è blu"`), oppure scegliendo opportunamente i delimitatori esterni quando possibile.
- Nota: l'interprete rileva la fine della stringa in base al delimitatore usato; un apice interno non preceduto da \ chiuderà prematuramente la stringa provocando un errore di sintassi.

Escape (backslash) e gestione dei caratteri speciali

- Backslash \: inserendolo prima di un apice o di una virgoletta all'interno della stringa si indica che quel carattere fa parte del contenuto della stringa e non è il delimitatore. Esempio: `testo = 'Paverino apre l\'ombrello se piove'` o `testo = "Paverino apre l\"ombrello\" se piove"`.

Usare l'escape è la soluzione quando non si possono cambiare i delimitatori o quando la stringa deve contenere entrambi i tipi di virgolette.

Variabili, assegnazione e print

- Assegnazione: la sintassi è `nome_variabile = valore`. Il nome a sinistra dell' '=' non va tra virgolette ed è l'etichetta della scatola in memoria. Esempio: `testo_da_visualizzare = "Paperino va al mare"`.

Nomi validi: si possono usare underscore nei nomi (es. `testo_da_visualizzare`) e devono essere sequenze non nulle di caratteri consentiti.

Concatenazione in print: è comune usare `print()` per visualizzare stringhe e variabili concatenate con `+`.

- Esempio: ``testodavisualizzare = "Paperino va al mare" numero = 4 print("Il tuo testo è: " + testodavisualizzare) `` Questo concatena "Il tuo testo è: " con il contenuto di `testo_da_visualizzare` e lo mostra a schermo.

Dove e come eseguire Python sul proprio computer

- Opzioni di installazione: Installare Anaconda/Miniconda: distribuzione consigliata per data science con navigator, Spyder e gestore di pacchetti conda.
- Installare solo Python: spesso già presente su Linux/Mac; su Windows lo si scarica dal sito ufficiale.

Editor / IDE consigliati: usare un editor testuale avanzato facilita lo sviluppo di script Python. Esempi (con breve nota): | Editor / IDE | Note | | --- | --- | | **Spyder** | orientato alla scienza e all'uso "puro" di Python; spesso incluso in Anaconda. | | **Visual Studio Code** | leggero, estendibile, molto usato; buone estensioni Python. | | **PyCharm** | IDE completo per Python con molte funzionalità (edizione Community disponibile). | | **Notepad++** | editor testuale semplice e leggero per Windows (meglio di Notepad). |

Evitare Notepad di base su Windows: non gestisce bene Unicode e può creare problemi con codifiche di file di testo.

Creare, salvare ed eseguire un programma Python

- File di testo con estensione `.py`: un programma Python è un file di testo che contiene istruzioni Python e deve essere salvato con estensione `.py` (es. `il_mio_primo_programma.py`).
- Visualizzare le estensioni su Windows: in Esplora risorse andare su Visualizza → Mostra → attivare Estensioni nomi file per vedere `.py`.
- Flusso di lavoro: Aprire un editor, scrivere le istruzioni Python (assegnazioni, `print`, ecc.).
- Salvare il file con `.py`.

- Aprire il terminale / prompt dei comandi (cmd su Windows, Terminal su macOS/Linux).
- Posizionarsi nella cartella che contiene il file (cd percorso) e lanciare lo script con python nomefile.py (o con il comando appropriato all'ambiente).
- Con Anaconda si può anche eseguire direttamente dall'IDE (Spyder) o usare il conda navigator per avviare ambienti e strumenti.

Note pratiche e suggerimenti rapidi

- Digitando il nome di una variabile nella REPL (interprete interattivo), Python mostra la rappresentazione stringa spesso usando gli apici singoli come delimitatori nella visualizzazione.
- Righelli bianchi (linee vuote) tra istruzioni sono ignorati dall'interprete; servono solo per leggibilità.
- Quando si lavora con testi che contengono caratteri non ASCII, preferire un editor che supporti Unicode per evitare problemi di codifica.

Salvare ed eseguire un programma Python

- File .py: un programma Python è semplicemente un file di testo con estensione .py che contiene istruzioni in Python.
- Flusso di sviluppo: scrivi nel editor → salvi il file → esegui nel terminale → modifichi → salvi → riesegui (ripeti). Questo è il ciclo standard anche se usi IDE grafici come VS Code, PyCharm, Spyder.

Eseguire da terminale

Comando/azione	Descrizione
Aprire il terminale (Windows: cmd o Terminal)	Interfaccia testuale equivalente alla visualizzazione a icone delle cartelle.
cd nome_cartella	Cambia la directory corrente (es. cd Desktop per andare sul Desktop).
dir / ls	Visualizza il contenuto della directory (Windows: dir, Linux/macOS: ls).
python nomefile.py	Esegue lo script Python nomefile.py nella directory corrente.

- Essere nella cartella che contiene il file è necessario per eseguirlo con python nomefile.py.
- Eseguendo il file, Python legge le istruzioni e la CPU le esegue; l'istruzione più comune per mostrare testo è print.

Input e Output

- Output: la funzione `print(...)` mostra testo a schermo (dal computer all'umano).
- Input: la funzione `input(...)` chiede dati all'utente (dall'umano al computer). Visualizza una stringa di prompt come fa `print`, poi si mette in attesa che l'utente scriva e prema Invio; il valore restituito è sempre una stringa.

Esempio d'uso:

- `nome = input("Come ti chiami? ")` — visualizza il prompt, attende l'input e memorizza la stringa in `nome`.
- `print("Mi chiamo", nome)` vs `print("Mi chiamo " + nome)` — entrambi mostrano il nome, ma operano diversamente (vedi sotto).

Differenze tra virgola e `+` in `print` e assegnazioni

Caso	Comportamento
<code>print("Mi chiamo", nome)</code>	<code>Print</code> mostra i due elementi separati da uno spazio automaticamente; la virgola è un separatore per il <code>print</code> .
<code>print("Mi chiamo " + nome)</code>	Il <code>+</code> concatena stringhe creando una nuova stringa; qui puoi controllare gli spazi manualmente.
<code>variabile = a, b</code>	Creazione di una tuple, non di una stringa. La virgola fuori dal <code>print</code> non concatena le stringhe.
<code>variabile = a + " " + b</code>	Produce una nuova stringa concatenata con uno spazio.

- La virgola in `print` non è un operatore di concatenazione: dentro `print` separa argomenti e il risultato visualizzato ha spazi automatici; fuori da `print` la virgola crea tuple.
- Per creare una singola stringa a partire da due stringhe si usa `+` o metodi come `f"..."` o `.join()`.

Tipi e conversioni (type casting)

- Input è stringa: tutta l'input letta con `input()` è di tipo `str`. Per fare operazioni numeriche bisogna convertire: `n = int(input("Inserisci un intero: "))` → converte la stringa in `int`.
- `x = float(input(...))` → converte in `float`.

Esempio pratico richiesto dagli esercizi: leggere due interi, calcolare somma e prodotto e mostrare anche i tipi:

- Leggere: `a = int(input("Primo intero: "))`, `b = int(input("Secondo intero: "))`.

- Calcoli: $somma = a + b$, $prodotto = a * b$.
- Visualizzare: `print("La somma è", somma, ", tipo", type(somma))` oppure usando concatenazione/format.

Se non converti, 5 e 7 letti come stringhe non potranno essere sommati come numeri (risulterà concatenazione o errore).

Piccolo promemoria su operatori con stringhe

- Concatenazione (+): `s1 + s2` crea una nuova stringa unendo `s1` e `s2`.
- Moltiplicazione (*): `s * n` ripete la stringa `s` per `n` volte (es. `"ha" * 3` → `"hahaha"`). È l'analogo della somma ripetuta per numeri.

Ambiente di sviluppo e strumenti

- IDE/Editor grafici: Visual Studio Code, PyCharm, Spyder offrono esecuzione grafica e debugging; comunque il meccanismo sottostante è sempre `edit` → `save` → `run`.
- Spyder: richiede Python installato prima; si trova spesso installando Anaconda (che include Spyder).
- Installazione Python: scaricare da python.org e installare (Windows/macOS).
- Anaconda: distribuzione che facilita l'installazione di Python, Spyder e altri pacchetti; procedere con le opzioni predefinite.
- Editor online: si possono usare editor nel browser (es. PythonAnywhere) per programmare da tablet o smartphone; alcuni servizi gratuiti richiedono registrazione.

Modalità interattiva (REPL)

- La modalità interattiva di Python consente di eseguire singole istruzioni e vedere subito i risultati (utile per test veloci).
- Esempio di esplorazione tipi e concatenazione: definire `t1 = "ciao"`, `t2 = "a tutti"`, provare `print(t1, t2)` vs `print(t1 + t2)` e usare `type(...)` per controllare il tipo delle variabili.

Esercizi pratici suggeriti

1. Programma che chiede due numeri interi, calcola e visualizza: • la somma e il prodotto, • il tipo di ciascun risultato (usa `type()` e conversione con `int()`). 2. (Opzionale) Esercizi sulla divisione e sulla conversione di tipi (attenzione alle divisioni che danno float). 3. (Più avanzato) Lavorare con stringhe: concatenazione, ripetizione e operatori.

Consigli pratici

- Se non hai il computer in aula, puoi svolgere molti esercizi su carta; la logica è importante tanto quanto l'esecuzione.
- Per chi inizia: imparare prima il ciclo edit-save-run e poi concentrarsi su cosa scrivere dentro il programma (algoritmi, strutture dati, sintassi) è la strada giusta.

Conversioni di tipo in Python

Concetto chiave

- `input()` restituisce sempre una stringa indipendentemente da quello che l'utente digita; quindi quando si legge un numero con `input()` il valore è una stringa (es. "5"), non un intero. Usare la conversione esplicita per ottenere il tipo desiderato.

Effetto sulle operazioni (concatenazione vs addizione)

- Stringhe + Stringhe: "5" + "7" produce "57" (concatenazione).
- Interi + Interi: 5 + 7 produce 12 (somma numerica).
- Se si ottiene "57" quando ci si aspettava 12, significa che una o entrambe le variabili sono stringhe.

Controllo del tipo

- Usare `type(x)` per verificare il tipo di una variabile. Utile per debuggare errori di concatenazione/somma.

Funzioni di conversione principali

Funzione	Converte in	Note ed esempio
<code>int(...)</code>	intero	<code>int("5")</code> → 5. Fallisce se la stringa non è un intero valido come "ciao" o "3.141".
<code>float(...)</code>	float (decimale)	<code>float("3.141")</code> → 3.141. Utile per numeri con virgola/punto.
<code>str(...)</code>	stringa	<code>str(5)</code> → "5". Serve per concatenare o stampare valori.

Usare la funzione corretta in base al formato testuale che si riceve.

Composizione di funzioni e parentesi (ordine di valutazione)

- È comune annidare conversioni, ad esempio `int(input(...))`: prima viene eseguito `input(...)` (restituisce una stringa), poi il risultato viene passato a `int(...)` per la conversione.
- Le parentesi hanno significato funzionale: `f(g(x))` significa passare il risultato di `g(x)` come argomento a `f`.
- L'ultimo livello di conversione determina il tipo finale della variabile. Esempio: se applichi `int(float(str(int(input(...)))))` il tipo finale sarà `int` (l'ultima conversione prevale).

Esempio pratico

- `x = input("Inserisci un intero: ")` → `x` è stringa, anche se l'utente digita 100.
- `x = int(input("Inserisci un intero: "))` → `x` è `int`, purché l'utente inserisca un valore convertibile (es. 100).
- Se l'utente digita "ciao" e si prova `int("ciao")`, si genera un errore perché la stringa non rappresenta un intero in base 10.

Errori comuni e come evitarli

- `ValueError`: succede quando si tenta di convertire una stringa non numerica in `int` o `float` (es. `int("ciao")` o `int("3.141")`). Soluzioni: validare l'input prima della conversione, usare `float()` per numeri con decimali, o gestire con `try/except`.

Parentesi non bilanciate: annidando molte funzioni è facile dimenticare una parentesi; contare le parentesi o scriverle con ordine aiuta a evitare errori di sintassi.

Verificare sempre con `type()` se si è incerti sul tipo prima di eseguire operazioni che richiedono tipi specifici.

Buone pratiche e consigli per esercizi

- Quando si chiede "inserisci un numero", convertire immediatamente con `int(input(...))` o `float(input(...))` a seconda del tipo previsto.
- Se l'input può essere sbagliato, usare una procedura di controllo: controllare con metodi di validazione;
- oppure usare `try/except ValueError` per richiedere di nuovo l'input.

Per debug veloce: stampare `type(variabile)` dopo l'assegnazione per capire che tipo è effettivamente il valore. 

Dove approfondire

- Le slide del corso contengono una sezione dedicata alle conversioni tra tipi con esempi pratici e casi d'uso; cercare la parte sulle conversioni se servono esempi aggiuntivi o chiarimenti. 